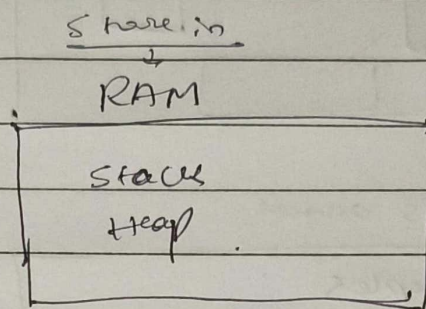
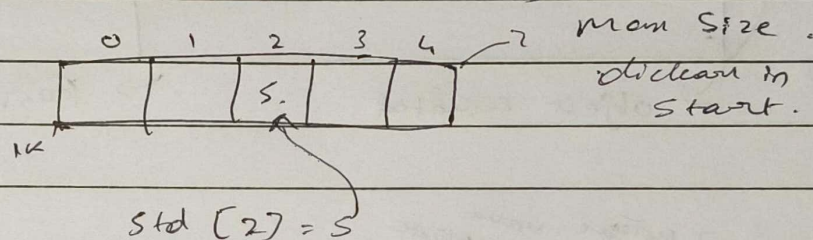


B Tuples in pythonData structures Lists, Tuples, Sets, Dictionaries

→ ? Inform² Memory ~ Store
→ Access.



Lists / Arrays.



RAM

DSA Studint :-

- Design
- memory allocation
- operations

- Add
- Remove
- Read
- Search
- Sort
- Append
- Insert at position

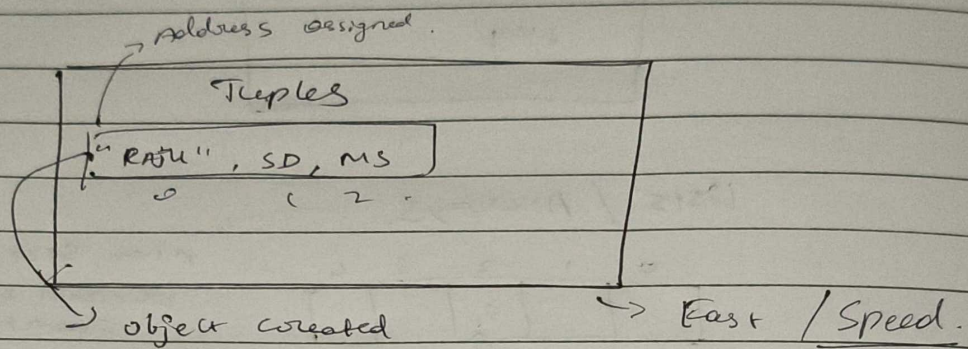
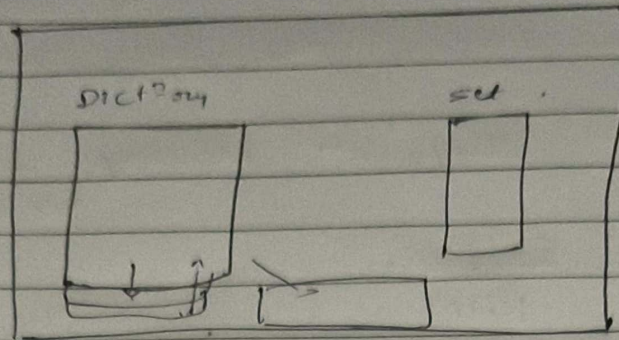
Time Complexity $O()$

→ mutable / immutable.

→ mutable

→ not change

RAM



unique value
no duplicate

Sets

$O(1)$
Search

10
50
80
90
110

List $O(n)$

10
50
80
90
110

* Dictionary :- $O(1)$ Search (high performance)

Key, Value.

ID, Name, Age, Course.

6_Data Structure in Python > 08_Tuples > 01_Mini Project.py > main

```
1  # Student Records System using Tuples
2  student_db = []
3
4  def add_student():
5      id = int(input("Enter Id"))
6      name = input("Enter name")
7      branch = input("enter branch")
8      year = int(input("Enter year"))
9      record = (id,name,branch,year)
10     student_db.append(record)
11
12 def display():
13     if not student_db:
14         print("No records available")
15
16     for student in student_db:
17         print(f"ID:{student[0]}, Name:{student[1]}, Branch:{student[2]} year:{student[3]}")
18
19 def search_by_id():
20     search_id = int(input("Enter ID to search: "))
21     found = False
22     for student in student_db:
23         if student[0] == search_id:
24             print(f"✅ Found: {student}")
25             found = True
26             break
27     if not found:
28         print("❌ Student not found.")
29
```

```

0 def main():
1     while True:
2         print("\n1. Add Student  2. Display All  3. Search by ID  4. Exit")
3         choice = input("Choose an Option: ")
4         if choice == '1':
5             add_student()
6         elif choice == '2':
7             display()
8
9         elif choice == '3':
10            search_by_id()
11
12        elif choice == '4':
13            print("exit")
14            break
15        else:
16            print("Invalid choice")
17            return
18    main()

```


05_Remove List Items.py 01_Creating tuples.py X 02_Access Tuple Items.py 03_Up

data Structure in Python > 08_Tuples > 01_Creating tuples.py > ...

```
# A tuple is a collection which is ordered and unchangeable.  
thistuple = ("apple", "banana", "cherry")  
print(thistuple)  
  
# Allow Duplicates  
thistuple = ("apple", "banana", "cherry", "apple", "cherry")  
print(thistuple)  
  
# Tuple Length  
print(len(thistuple))  
  
# Create Tuple With One Item  
# One item tuple, remember the comma:  
# type()  
thistuple = ("apple",)  
print(type(thistuple))  
  
#NOT a tuple  
thistuple = ("apple")  
print(type(thistuple))  
  
# Tuple Items - Data Types  
# A tuple with strings, integers and boolean values:  
tuple1 = ("abc", 34, True, 40, "male")  
print(tuple1)  
  
# The tuple() Constructor  
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets  
print(thistuple)
```


05_Remove List Items.py

01_Creating tuples.py

02_Access Tuple Items.py X

03_U

Data Structure in Python > 08_Tuples > 02_Access Tuple Items.py > ...

```
1 # Access Tuple Items
```

```
2 thistuple = ("apple", "banana", "cherry")
```

```
3 print(thistuple[1])
```

```
4  
5 # Negative Indexing
```

```
6 print(thistuple[-1])
```

```
7  
8 # Range of Indexes
```

```
9 thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
```

```
0 print(thistuple[2:5])
```

```
1  
2 # This example returns the items from the beginning to, but NOT included, "kiwi":
```

```
3 print(thistuple[:4])
```

```
4  
5 # This example returns the items from "cherry" and to the end:
```

```
6 print(thistuple[2:])
```

```
7  
8 # Range of Negative Indexes
```

```
9 print(thistuple[-4:-1])
```

```
0  
1 # Check if Item Exists
```

```
2 if "apple" in thistuple:
```

```
3     print("yes")
```


6_Data Structure in Python > 08_Tuples > 03_Update Tuples.py > ...

```
1  # Change Tuple Values
2  # Convert the tuple into a list to be able to change it:
3  tuple1 = ("apple", "banana", "cherry")
4  Update_tuple = list(tuple1)
5  Update_tuple[1] = "kiwi"
6  tuple1 = tuple(Update_tuple)
7  print(tuple1)
8
9  # Add Items
10 Update_tuple.append("orange")
11 tuple1 = tuple(Update_tuple)
12 print(tuple1)
13
14 # . Add tuple to a tuple.
15 thistuple = ("apple", "banana", "cherry")
16 y = ("orange",)
17 thistuple += y
18 print(thistuple)
19
20 # Remove Items
21 Update_tuple.remove("apple")
22 tuple1 = tuple(Update_tuple)
23 print(tuple1)
24
25 # The del keyword can delete the tuple completely:
26 del thistuple
27 print(thistuple) #this will raise an error because the tuple no longer exists
28
```

6_Data Structure in Python > 08_Tuples > 04_Unpack Tuples.py > ...

```
1  # Unpacking a Tuple
2  fruits = ("apple", "banana", "cherry")
3  x,y,z = fruits
4  print(x)
5  print(y)
6  print(z)
7
8  # Using Asterisk*
9  # Assign the rest of the values as a list called "red":
10 fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
11
12 (green, yellow, *red) = fruits
13
14 print(green)
15 print(yellow)
16 print(red)
17
18 (green, *tropic, red) = fruits
19 print(green)
20 print(tropic)
21 print(red)
```


05_Remove List Items.py

01_Creating tuples.py

02_Access Tuple Items.py

03_

6_Data Structure in Python > 08_Tuples > 05_Loop Tuples.py > ...

```
1  # Loop Through a Tuple
2  thistuple = ("apple", "banana", "cherry")
3  for x in thistuple:
4      print(x)
5
6  # Loop Through the Index Numbers
7  for i in range(len(thistuple)):
8      print(thistuple[i])
9
10 # Using a While Loop
11 i = 0
12 while i < len(thistuple):
13     print(thistuple[i])
14     i = i + 1
```


6_Data Structure in Python > 08_Tuples > 06_Join Tuples.py > ...

```
1  # Join Two Tuples
2  tuple1 = ("a", "b" , "c")
3  tuple2 = (1, 2, 3)
4
5  tuple3 = tuple1 + tuple2
6  print(tuple3)
7
8  # Multiply Tuples
9  fruits = ("apple", "banana", "cherry")
10 mytuple = fruits * 2
11
12 print(mytuple)
13
```

_Access Tuple Items.py 03_Update Tuples.py 04_Unpack Tuples.py 05_Loop Tuple

Data Structure in Python > 08_Tuples > 07_Tuple Methods.py > ...

```
1  # Python Tuple count() Method
2  thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
3  x = thistuple.count(5)
4  print(x)
5
6  # Python Tuple index() Method
7  # Search for the first occurrence of the value 8, and return its position:
8  thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
9  x = thistuple.index(8)
10 print(x)
```